Conception et Exploitation d'une BDD

SQL Server • TP n°3 : Fonctions, curseurs et triggers

Contexte Professionnel

Vous intervenez en tant que développeur SQL pour une plateforme spécialisée dans la location de véhicules courte et moyenne durée. Actuellement, l'application cliente prend en charge l'essentiel de la logique métier. L'équipe technique a décidé de migrer une partie de cette intelligence directement au niveau du moteur de base de données (SQL Server).

OBJECTIFS DE LA MIGRATION

- **Centraliser** les calculs financiers complexes.
- **Automatiser** les opérations récurrentes et les contrôles de conformité.
- **Historiser** toutes les modifications critiques pour des raisons d'audit.
- **Sécuriser** les données métier en empêchant les manipulations frauduleuses ou erronées au plus bas niveau.

1. Schéma de la Base de Données

Le système repose sur quatre entités principales gérant l'activité quotidienne :

Entité	Rôle et Données Principales
CLIENT	Identité des locataires (Identifiant, Nom, Prénom, Email).
VEHICULE	Parc automobile (Immatriculation, Marque, Modèle, Catégorie, Prix journalier, Kilométrage actuel, Statut de disponibilité).
LOCATION	Contrats de location liant un client à un véhicule sur une période (Dates de début/fin, Prix total, Statut de la location).
ENTRETIEN	Suivi de la maintenance des véhicules (Date, Type d'opération, Coût, Kilométrage relevé).

Note : Les statuts des véhicules varient généralement entre `DISPONIBLE`, `LOUE`, et `ENTRETIEN`. Les locations évoluent entre `RESERVEE`, `EN COURS`, `TERMINEE` ou `ANNULEE`.

Exercice 1 : Calcul du prix d'une location

BESOIN MÉTIER

L'application calcule actuellement le prix des locations de façon redondante dans plusieurs modules. L'équipe souhaite uniformiser cette règle grâce à une **fonction scalaire** intégrée à la base de données. Cette fonction doit calculer le prix théorique d'une location en se basant sur le prix journalier du véhicule multiplié par le nombre de jours effectifs (déduit des dates de début et de fin).

TRAVAIL ET ANALYSE ATTENDUS

- Concevoir la fonction scalaire prenant en paramètre les dates et le tarif journalier, et retournant le montant total.
- Mettre à l'épreuve cette fonction en la combinant à une requête de consultation globale des locations.
- **Analyse de cohérence :** Comparer le prix calculé dynamiquement avec le prix historiquement figé dans la table `LOCATION` afin de détecter d'éventuelles anomalies de facturation passées.

Exercice 2 : Recherche dynamique de disponibilité

BESOIN MÉTIER

Le service commercial a besoin d'un moteur de recherche flexible pour identifier les véhicules répondant aux critères des clients (catégorie souhaitée, budget journalier maximum, statut actuel). Tous ces critères sont optionnels.

> ### APPROCHE TECHNIQUE : FONCTION RETOURNANT UNE TABLE
>
> La solution optimale consiste à développer une fonction de type *Table-Valued Function (TVF)*. Elle encapsule la logique de filtrage complexe et retourne un ensemble de résultats interrogeable comme une table standard.
>
> **Avantage majeur :** Contrairement à une procédure stockée classique, une TVF peut être intégrée directement dans la clause `FROM` ou `JOIN` d'une requête, permettant de combiner ses résultats avec d'autres tables (pour afficher le nom de la catégorie ou l'historique d'entretien par exemple).

Le développement devra permettre des appels modulaires, par exemple : filtrer uniquement les SUV disponibles, ou croiser les filtres pour obtenir les citadines à moins de 80 € par jour.

Exercice 3 : Contrôle de la flotte (Curseurs)

CONTEXTE ET CLASSIFICATION

Le responsable de la flotte procède à des audits réguliers. Le niveau d'attention requis pour chaque véhicule dépend strictement de son kilométrage accumulé :

Kilométrage relevé	Niveau de contrôle affecté
Inférieur à 20 000 km	**NORMAL**
De 20 000 km à 50 000 km	**SURVEILLANCE**
Supérieur à 50 000 km	**PRIORITAIRE**

IMPLÉMENTATION VIA CURSEUR

Il est demandé de simuler un traitement séquentiel en implémentant un **curseur**. Le script devra parcourir la flotte ligne par ligne, évaluer le kilométrage de chaque véhicule et lui attribuer son niveau d'alerte avant de passer au suivant.

Analyse critique : Bien que l'exercice impose l'utilisation d'un curseur pour la simulation d'un traitement lourd, cette classification précise aurait pu être résolue de manière beaucoup plus performante par une simple requête ensembliste exploitant une expression conditionnelle de type `CASE WHEN ... THEN ...`. Les curseurs doivent généralement être réservés aux cas où l'itération ligne à ligne est incontournable (appels d'API externes, traitements dépendants de l'état précédent).

Exercice 4 : Historisation des changements (Triggers)

CONTEXTE D'AUDIT

Pour des raisons de suivi et d'assurance, toute modification d'un véhicule doit laisser une trace, en particulier les changements de statut et les mises à jour kilométriques.

STRATÉGIE DE DÉCLENCHEMENT (TRIGGER)

Le mécanisme idéal est un **Trigger AFTER UPDATE** placé sur la table `VEHICULE`. Ce déclencheur capte la transaction de mise à jour, compare les anciennes valeurs (accessibles via la table virtuelle `deleted`) et les nouvelles valeurs (via `inserted`), puis insère automatiquement une ligne d'archive dans une nouvelle table `HISTORIQUE_VEHICULE` si et seulement si les champs surveillés ont réellement changé.

Exercice 5 : Sécuriser le kilométrage

RÈGLE MÉTIER STRICTE

Une donnée kilométrique est par nature incrémentale. Le système doit garantir qu'il est physiquement impossible de diminuer le compteur d'un véhicule (sauf intervention exceptionnelle non traitée ici). Passer de 42 500 à 43 100 est autorisé ; passer de 44 000 à 42 000 doit être rejeté par la base de données elle-même.

MISE EN ŒUVRE

Un trigger préventif doit intercepter l'instruction de mise à jour. En croisant les tables d'état (avant/après), il repère toute tentative de diminution. Si une infraction est détectée, le trigger lève une exception (via une instruction de type `THROW`) et fait échouer la transaction entière, protégeant ainsi l'intégrité de la flotte.

Exercice 6 : Analyse d'un Trigger existant

LE PROBLÈME DES MISES À JOUR ENSEMBLISTES

Un développeur junior a écrit un trigger de contrôle de kilométrage en utilisant des variables scalaires pour stocker l'identifiant du véhicule, son ancien kilométrage et son nouveau kilométrage, extraits directement des tables système temporaires.

ANALYSE DU DÉFAUT DE CONCEPTION

L'erreur fondamentale de ce script est qu'il suppose qu'une mise à jour ne concerne **qu'une seule ligne à la fois**. En SQL Server, un trigger s'exécute une seule fois par instruction (par "batch"), même si celle-ci modifie 1000 véhicules d'un coup.

En assignant les résultats de `inserted` et `deleted` à des variables uniques, le trigger n'évalue qu'une seule ligne arbitraire du lot. Si d'autres lignes contiennent des anomalies (des kilométrages réduits), elles passeront inaperçues.

SOLUTION EXIGÉE

La correction implique l'abandon des variables isolées au profit d'une logique ensembliste : il faut lier (jointure) les ensembles `inserted` et `deleted` sur l'identifiant du véhicule, et vérifier si au moins un enregistrement dans cet ensemble présente une incohérence kilométrique.

Exercice 7 : Choisir le bon mécanisme T-SQL

Faisons le bilan des concepts abordés. À chaque besoin métier correspond une réponse architecturale optimale dans le moteur de base de données.

Situation métier	Mécanisme adapté & Justification
A - Calculer automatiquement le prix d'une location à partir de plusieurs valeurs.	**Fonction scalaire :** Parfaite pour encapsuler une formule de calcul prenant des paramètres et retournant une valeur unique (le prix), facilement intégrable dans un simple SELECT.
B - Retourner la liste des véhicules disponibles correspondant à plusieurs critères.	**Fonction retournant une table (TVF) :** Idéale pour renvoyer un jeu de résultats filtré. Contrairement à une procédure, elle peut être combinée avec d'autres tables via des jointures.
C - Déclencher l'enregistrement d'un historique lorsqu'un véhicule est modifié.	**Trigger (Déclencheur) :** C'est le seul mécanisme capable d'écouter passivement les événements de modification (DML) et d'y réagir de manière autonome et transparente pour garantir l'audit.
D - Exécuter explicitement une opération métier pour clôturer une location.	**Procédure stockée :** Recommandée pour orchestrer une suite d'actions (modifier la location, libérer le véhicule, générer une facture) au sein d'une transaction maîtrisée appelée par l'application.
E - Parcourir une liste où le traitement dépend de l'état calculé pour l'élément précédent.	**Curseur :** Nécessaire lorsque la logique relationnelle ensembliste classique ne suffit pas et qu'un traitement séquentiel strict (ligne par ligne avec dépendance temporelle) s'impose.